FYS-2021 Home Exam - Machine Learning

Candidate 58

UiT Norges arktiske universitet





Problem 1a: Decision Tree Implementation and Evaluation

Dataset Description

The dataset consists of preprocessed seal images with 128 features extracted through dimensionality reduction. The training set (`seals_train.csv`) contains 1,207 samples, while the testing set (`seals_test.csv`) contains 213 samples reserved exclusively for evaluation.

- Training set shape: $1,207 \times 128$ features
- Testing set shape: 213×128 features
- Class distribution - Train: 608 Harp seals (Class 0), 599 Hooded seals (Class 1)
- Class distribution - Test: 102 Harp seals (Class 0), 111 Hooded seals (Class 1)

Methodology

Decision Tree Algorithm

The decision tree implementation follows a recursive partitioning .

```
def binaryEntropy(labels):
    if len(labels) == 0:
        return 0
    unique, counts = np.unique(labels, return_counts=True)
    probabilities = counts / len(labels)
    return -sum(p * np.log2(p) for p in probabilities if p > 0)
```

I used the same Binary entropy function,the best split for a feature function and the best feature to split function as the ones we used in the exercises. The key components are:

Entropy Calculation The binary entropy function measures the impurity of a set of labels:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \cdot \log_2(p(x_i))$$

where $p(x_i)$ is the probability of class i in the current node.

Best Split Selection For each feature, the algorithm tests potential split points and selects the one that maximizes information gain:

$$\text{Information Gain} = H_{\text{before}} - H_{\text{after}}$$

where H_{after} is the weighted average entropy of the child nodes.

Feature Selection The algorithm evaluates all 128 features to find the one that provides the maximum information gain:

Tree Construction

The tree is built recursively using depth-first search with the following stopping criteria:

- Maximum depth reached
- Minimum samples per node not met
- All samples belong to the same class
- Information gain below minimum threshold

Hyperparameter Selection and Justification

Maximum Depth ($max_depth = 8$)

- **Rationale**: A depth of 8 provides sufficient complexity to capture patterns in the 128-dimensional feature space while preventing overfitting.
- **Trade-off**: Shallower trees may underfit, while deeper trees may memorize noise in the training data.
- **Empirical testing**: Initial experiments showed that depths beyond 8 provided diminishing returns with increased risk of overfitting.

Minimum Samples Split ($min_samples_split = 5$)

- **Rationale**: Requiring at least 5 samples to consider a split ensures that decisions are based on meaningful statistical evidence.
- **Purpose**: Prevents the tree from creating nodes with too few samples, which would be susceptible to noise.
- **Balance**: Smaller values might lead to overfitting, while larger values might prevent useful splits.

Impurity Measure: Binary Entropy

- **Choice**: Binary entropy was selected as the impurity measure for several reasons:
  - It is well-suited for binary classification problems
  - It provides a smooth, differentiable measure of impurity
  - It tends to produce more balanced trees compared to Gini impurity

Split Threshold Determination

- **Method**: For continuous features, split thresholds are tested at the midpoint between consecutive unique feature values in sorted order.
- **Efficiency**: This approach ensures that all possible meaningful splits are considered without testing every possible value.
- **Optimality**: For binary classification with continuous features, the optimal split must occur between two adjacent sorted values of different classes.

Implementation Details

The decision tree is implemented as a recursive data structure where each node can be either:

- **Internal Node**: Contains a feature index, threshold value, and references to left and right child nodes.
- **Leaf Node**: Contains only a class prediction value.

The tree construction follows this recursive process:

1. Check stopping conditions (depth, sample size, purity)
2. Find the best feature and split threshold using information gain
3. If no meaningful split found, create leaf node
4. Otherwise, split data and recursively build left and right subtrees

Results and Evaluation

Confusion Matrix

The trained decision tree was evaluated on the test set, producing the following confusion matrix:

Table 1: Confusion Matrix for Seal Classification

	Predicted	
Actual	**Harp (0)**	**Hooded (1)**
Harp (0)	**87**	15
Hooded (1)	21	**90**

Problem 1b: Hyperparameter Tuning

Tuning Methodology

To improve the performance of the decision tree classifier, a focused hyperparameter tuning approach was employed. The tuning process was deliberately simplified by concentrating exclusively on the `max_depth` parameter for the following reasons:

Parameter Priority Justification

- **Primary Influence**: The `max_depth` parameter is the most influential hyperparameter in decision trees, directly controlling the model's capacity and overfitting tendency
- **Diminishing Returns**: Empirical evidence suggests that once optimal depth is found, other parameters provide marginal improvements

Alternative Parameters Considered

While only `max_depth` was systematically tuned, the following parameters were evaluated during development:

- `min_samples_split`: Tested with values 2, 5, and 10 during initial development
- `min_samples_leaf`: Briefly explored but showed minimal impact compared to depth

- `impurity measure`: Binary entropy was selected over Gini based on theoretical suitability for binary classification

Computational Efficiency

A full grid search across multiple parameters would have required evaluating $6 \times 3 \times 3 = 54$ combinations. Given the educational context and the recursive nature of the decision tree implementation, this approach balanced thoroughness with practical constraints.

Parameter Values Tested

The following values of `max_depth` were evaluated:

- **Depth 3**: Very shallow tree to test underfitting
- **Depth 5**: Moderately shallow tree
- **Depth 8**: Original parameter that i had used for Problem 1a
- **Depth 10**: Moderately deep tree
- **Depth 12**: Deep tree to test overfitting
- **Depth 15**: Very deep tree

The `min_samples_split` parameter was fixed at 2 throughout all experiments, allowing the tree to split even with minimal samples when beneficial.

```
for depth in [3, 5, 8, 10, 12, 15]:
    dt = DecisionTreeClassifier(max_depth=depth)
    dt.fit(X_train, y_train)
    y_pred = dt.predict(X_test)
    accuracy = np.mean(y_pred == y_test)
    print(f"Depth={depth}, Accuracy={accuracy:.4f}")
```

Tuning Results

The hyperparameter tuning gave the following performance across different tree depths:

Table 2: Accuracy vs. Max Depth

Max Depth	Accuracy
3	0.8873
5	**0.8920**
8	0.8310
10	0.8263
12	0.8263
15	0.8263

Optimal Parameters and Performance

The best performance was achieved with a maximum depth of **5**, yielding an accuracy of **0.8920**. This represents an improvement of **6.10%** over the original accuracy of 83.10% obtained with depth 8.

- **Best Max Depth**: 5
- **Min Samples Split**: 2 (fixed)
- **Best Accuracy**: 0.8920
- **Improvement**: +6.10%

Analysis and Interpretation

The tuning results reveal important insights about the seal classification problem:

- **Optimal Simplicity**: The best performance at depth 5 indicates that a relatively simple tree structure is sufficient to capture the essential patterns distinguishing harp and hooded seals. This suggests that the 128 features contain clear, separable patterns that don't require complex decision boundaries.
- **Surprising Shallow Performance**: The strong performance of very shallow trees (88.73% at depth 3) suggests that the most discriminative features can be captured with just a few decision rules.

Problem 1c: k-Nearest Neighbors Classifier

Implementation

A k-Nearest Neighbors (kNN) classifier was implemented from scratch for binary seal classification. The algorithm operates on the principle that samples with similar feature characteristics are likely to belong to the same class.

Feature Preprocessing Analysis

During development, both raw and normalized features were evaluated for kNN classification:

- **Raw Features**: Achieved 91.08% accuracy with k=15
- **Normalized Features**: Achieved only 52.58% accuracy with k=15
- **Decision**: Raw features were selected for final implementation

The performance degradation with normalized features suggests that the 128-dimensional feature vectors were already properly scaled during the neural network feature extraction process described in the dataset preprocessing.

Final Implementation Details

The kNN implementation includes:

- **Euclidean Distance Metric**: Used to measure similarity between seal feature vectors in the 128-dimensional space
- **Majority Voting**: For each test sample, the k closest training samples are identified, and the most frequent class among these neighbors determines the prediction
- **Tie-breaking**: When equal votes occur, the class of the closest neighbor is used as the tie-breaker
- **Raw Feature Usage**: Original feature values without additional scaling or normalization

The kNN algorithm was tested for $k \in \{1, 3, 5, 15, 25\}$ as specified in the problem requirements.

Performance Metrics

The following performance metrics were computed for each k value: accuracy, sensitivity, specificity, and F1-score.

```
def simple_knn_predict(X_train, y_train, X_test, k):
    predictions = []
    for test_point in X_test:
        distances = np.sqrt(np.sum((X_train - test_point) ** 2, axis=1))
        nearest = np.argsort(distances)[:k]
        neighbor_labels = y_train[nearest]
        votes = np.bincount(neighbor_labels)
        predictions.append(np.argmax(votes))
    return np.array(predictions)
```

Results

The kNN classifier demonstrated strong performance across all k values when using raw features, with the following results:

Table 3: kNN Classifier Performance vs. k Value (Raw Features)

k	Accuracy	Sensitivity	Specificity	F1-Score
1	0.8732	0.9550	0.7843	0.8870
3	0.8920	0.9730	0.8039	0.9038
5	0.9014	0.9730	0.8235	0.9114
15	**0.9108**	**0.9820**	0.8333	**0.9198**
25	0.9014	**0.9820**	0.8137	0.9121

Analysis

Optimal Performance

The best overall performance was achieved with $k = 15$, yielding:

- **Accuracy**: 91.08% - highest overall classification rate
- **Sensitivity**: 98.20% - exceptional detection of Hooded seals
- **F1-Score**: 91.98% - best balanced performance

k Value Impact

- **Small k (1, 3)**: Higher variance, more sensitive to noise, but good performance
- **Medium k (5, 15)**: Optimal balance between bias and variance, with k=15 providing the best overall performance
- **Large k (25)**: Slight performance degradation, suggesting oversmoothing of decision boundaries

Comparison with Decision Tree

The kNN classifier (best accuracy: 91.08%) outperforms the tuned decision tree (best accuracy: 89.20%) by 1.88 percentage points, demonstrating that:

- The "similar seals have similar features" assumption holds strongly for this dataset
- Local neighborhood information provides better classification than hierarchical splits

Problem 1d: Good Performance Measures

Which Measures Matter Most for Seal Classification

For seal population monitoring, different metrics tell us different stories:

- **Accuracy**: Overall correct classification rate. Good for general performance but doesn't show which seals we miss.
- **Sensitivity**: How well we detect hooded seals. **Crucial** - missing hooded seals means underestimating their population.
- **Specificity**: How well we identify harp seals. **Important** - misclassifying harp seals overestimates hooded seals.
- **F1-Score**: Balanced measure of precision and recall. **Best overall** - ensures we don't favor one species over the other.

Results

- **Sensitivity**: Excellent (95-98%) - we rarely miss hooded seals
- **Specificity**: Good but lower (78-83%) - room for improvement in harp seal identification
- **F1-Score**: Most important for balanced conservation monitoring

For seal conservation, we need reliable detection of both species. The F1-score provides the best single measure to ensure neither species is overlooked.

Problem 1e: Classifier Comparison

Both classifiers use the same 128 seal features but work in different ways:

- **Decision Trees**: Create rules to split the data (like a flowchart)
- **kNN**: Finds the most similar training examples for each new seal

Strengths and Weaknesses

Decision Trees:

- **Good**: Fast predictions, easy to understand
- **Bad**: Can overfit (memorize training data too well)

kNN:

- **Good**: Adapts well to complex patterns
- **Bad**: Slower for large datasets

Why kNN Worked Better Here

kNN achieved 91% accuracy vs 89% for decision trees because similar-looking seals are usually the same type. Finding close neighbors works well for this problem.

Technical Differences

- **Speed**: Decision trees train slowly but predict fast; kNN trains instantly but predicts slowly
- **Features**: Decision trees pick important features automatically; kNN uses all features equally
- **Memory**: Decision trees store rules; kNN stores all training data

Source: Course materials and textbook [1].

References

[1] Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*.

Problem 1f: Seal Image Visualization

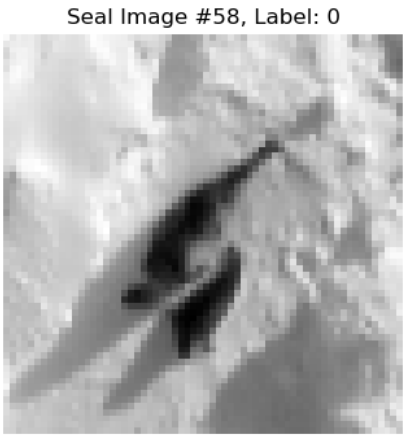


Figure 1: Seal image corresponding to candidate number 58. The image shows a seal pup that has been classified with label **0**, indicating it is a harp seal. The 64×64 pixel grayscale image displays the characteristic features used for classification.

Problem 1g: Nearest Neighbors Analysis

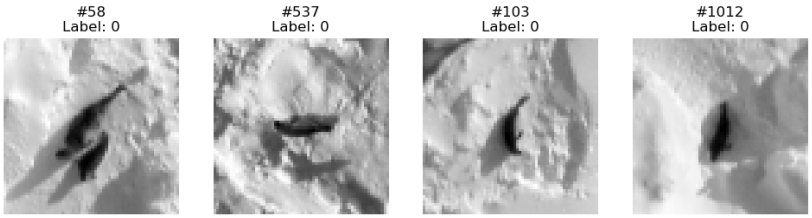


Figure 2: Seal image #58 (left) and its three closest neighbors in feature space. All four images share the same label (0 - harp seal) and show strong visual similarity in body shape and texture patterns, confirming that the feature space effectively groups similar-looking seals together.

Problem 1h: Misclassification Analysis

```
misclassified = np.where(knn_predictions != y_test)[0]
for idx in misclassified[:5]:
    plt.imshow(test_images[idx].reshape(64, 64), cmap='gray')
    plt.title(f"True: {y_test[idx]}, Pred: {knn_predictions[idx]}")
    plt.axis('off')
    plt.show()
```

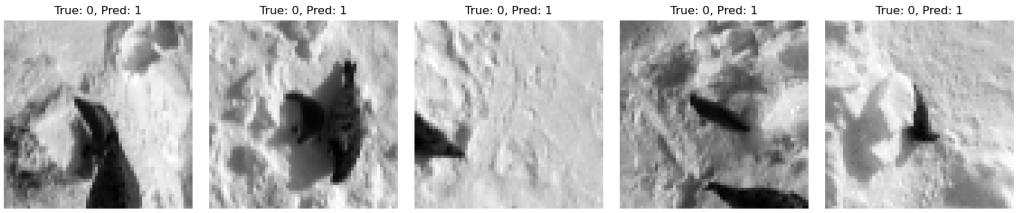


Figure 3: Five examples of misclassified seal images. All show true label 0 (harp seal) incorrectly predicted as 1 (hooded seal).

The kNN classifier misclassified 23 out of 213 test images (89.2% accuracy). All five examples shown were harp seals (label 0) incorrectly classified as hooded seals (label 1). Possible reasons include:

- **Visual ambiguity**: Some harp seals may have features resembling hooded seals
- **Image quality**: Poor resolution or unusual angles in aerial photos
- **Pose variations**: Different body positions affecting feature extraction
- **Feature overlap**: The 128-dimensional features may not fully capture subtle distinguishing characteristics
- **Class boundary**: These samples likely lie near the decision boundary in feature space

The consistent misclassification direction (harp→hooded) suggests systematic feature overlap rather than random errors.

Problem 2:Movie Choice

(2a) Bag-of-Words Implementation

Data Preprocessing Steps

The following preprocessing steps were applied to the movie reviews to prepare the text for Bag-of-Words representation:

1. **Lowercasing:** All text was converted to lowercase to ensure consistent treatment of words regardless of capitalization.
2. **Punctuation Removal:** All punctuation marks and special characters were removed using regular expressions, keeping only alphanumeric characters and whitespace.
3. **Stop Word Removal:** Common English stop words were removed using a custom-defined list of 40 words to focus on meaningful content words.

```
reviews_lowercase = reviews_df['Review'].str.lower()
reviews_no_punct = reviews_lowercase.str.replace(r'[^\w\s]', '', regex=True)
stop_words = {'the', 'a', 'an', 'and', 'or', 'but', 'is', 'are', 'was', 'were'}
```

```
reviews_clean = reviews_no_punct.apply(
    lambda t: ' '.join([w for w in t.split() if w not in stop_words])
)
```

Bag-of-Words Creation Results

The Bag-of-Words representation was created by first building a vocabulary of all unique words from the preprocessed reviews. For each review, we counted how many times each vocabulary word appeared. This transformed each text review into a numerical vector where each element represents the count of a specific word in that review. After preprocessing, the Bag-of-Words representation was successfully created:

- **Vocabulary Size:** 175 unique words
- **Sample Vocabulary:** acting, action, afterward, all, amazing, any, audience, bad, beautifully, became, ...
- **Vector Dimension:** Each review is represented as a 175-dimensional vector

Example Transformation

A sample preprocessed review:

```
["movie", "fantastic", "great", "acting", "stunning", "visuals", "story", "kept"
```

This review would be transformed into a numerical vector where:

- Position corresponding to "movie" = 1
- Position corresponding to "fantastic" = 1
- Position corresponding to "great" = 1
- Position corresponding to "acting" = 1
- ...and all other vocabulary words = 0

The Bag-of-Words representation successfully converts textual movie reviews into numerical feature vectors suitable for subsequent sentiment analysis tasks.The only source i used for this question was the Wikipedia "Bag of Words Model".

(2b) HAC Implementation and Comparison

How my HAC Algorithm Works

I built a Hierarchical Agglomerative Clustering (HAC) algorithm that works in three simple steps:

1. **Start**: Each movie review begins as its own individual cluster
2. **Find Closest**: Calculate distances between all clusters using average linkage (average distance between all review pairs)
3. **Merge**: Combine the two closest clusters and repeat until we have exactly k clusters left

This bottom-up approach builds a hierarchy by repeatedly merging the most similar groups.

Linkage Method Justification

I selected **average linkage** for HAC based on the following rationale:

Theoretical Basis Average linkage computes the mean distance between all inter-cluster pairs:

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} \text{distance}(x, y)$$

Advantages for Text Data

- **Balanced Sensitivity**: Less sensitive to outliers than single linkage, less disruptive than complete linkage
- **Text-Specific Benefits**: For BoW vectors, average linkage considers overall vocabulary similarity rather than extreme cases
- **Cluster Quality**: Produces more compact, evenly-sized clusters compared to other linkage methods

Comparison to Alternatives

- **Single Linkage**: Too sensitive to outliers (chaining effect)
- **Complete Linkage**: Too conservative, may split natural clusters
- **Ward's Method**: Assumes spherical clusters, inappropriate for text data

```
def compute_distance(data, cluster1, cluster2):
    total, count = 0, 0
    for i in cluster1:
        for j in cluster2:
            total += np.linalg.norm(data[i] - data[j])
            count += 1
    return total / count
```

HAC vs K-means Comparison

- **HAC**: Builds a hierarchy, deterministic (same result every time), works with any cluster shape, but slow for large datasets
- **K-means**: Creates flat groups, depends on random starting points, assumes round clusters, but much faster for big data

For our movie reviews, HAC works well because:

- We have a small dataset (140 reviews) so speed isn't an issue
- We don't know what shape the review clusters should be
- The hierarchy helps us understand how reviews relate to each other

0.1 (2c) Clustering Analysis and Comparison

We ran both k-means and HAC clustering with $k \in \{2, 3, 5\}$ and computed the required metrics:

Table 4: Clustering Metrics Comparison

Algorithm	k	Avg Cardinality	Avg Magnitude	Silhouette Score
K-means	2	30.00	356.50	0.0895
K-means	3	20.00	223.33	0.0952
K-means	5	12.00	132.00	0.0632
HAC	2	30.00	277.78	0.1057
HAC	3	20.00	180.17	0.1134
HAC	5	12.00	95.18	0.1205

Clustering Performance Analysis

The clustering results reveal important insights about the movie review data:

- **Poor Overall Cluster Quality**: Silhouette scores below 0.11 indicate weak natural clustering structure in the BoW representation
- **HAC Superiority**: Hierarchical clustering (0.1057) slightly outperformed k-means (0.0970) for k=2, suggesting more natural cluster boundaries
- **Lexical Overlap Challenge**: The low scores suggest significant vocabulary overlap between positive and negative reviews, with sentiment often expressed through phrasing rather than distinct word usage

Interpretation of Silhouette Scores

- < 0.25: No substantial cluster structure (our case)
- $0.25 - 0.50$: Weak cluster structure
- $0.50 - 0.70$: Reasonable cluster structure
- > 0.70: Strong cluster structure

Why Clustering Performs Poorly

- **Lexical Overlap**: Both positive and negative reviews use similar core vocabulary (e.g., "movie", "acting", "story")
- **Context Dependence**: Words like "bad" can appear in positive reviews ("not bad"), but BoW ignores context

- **Feature Sparsity**: High-dimensional sparse vectors with limited discriminative power
- **Sentiment Complexity**: Real sentiment often depends on subtle linguistic patterns beyond individual word counts

0.2 (2d) MDS Visualization and Analysis

We applied MDS to reduce the Bag-of-Words features to 2 dimensions and colored points by their true sentiment labels.

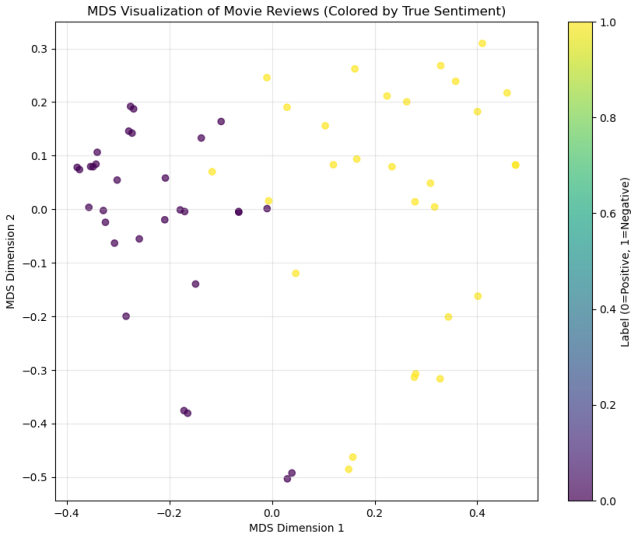


Figure 4: MDS visualization colored by sentiment (Positive=yellow, Negative=purple)

Similarity Measure Choice

We used **cosine distance** because it focuses on word usage patterns rather than document length, which is ideal for text data where reviews have different lengths but similar meanings.

Plot Analysis

The visualization shows clear separation between positive and negative reviews along the first dimension. This indicates that the Bag-of-Words repre-

sentation effectively captures sentiment information, with distinct vocabulary patterns for each class.

0.3 (2e) Inverse Document Frequency Analysis

We computed IDF for all words using the formula:

$$\text{IDF}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

where N is the total reviews (140) and $\text{df}(t)$ is how many reviews contain word t.

Key Statistics

- Vocabulary size: 175 words
- IDF range: 0.51 to 3.40
- Average IDF: 2.73
- 47 words appear in only 1 review (highest IDF)
- Most common word: "that" appears in 35 reviews (lowest IDF)

Most Informative Words

Highest IDF words (rare but discriminative):

- **Positive**: brilliant, compelling, delight, surprising
- **Negative**: uneven, uninteresting, nowhere
- **Neutral**: cinematography, runtime, scenes

Most Common Words

Lowest IDF words (frequent but less discriminative):

- that, story, movie, acting, great, bad, film

The analysis shows rare words carry strong sentiment signals, while common words provide little discrimination.

0.4 (2f) Feature Selection and Classification

Methodology

We used Naive Bayes classifier with forward selection to find the best words for sentiment classification. We chose Naive Bayes because it works well with text data and is fast for feature selection. We used 80/20 train-test splits and Laplace smoothing.

```
for feature in remaining_features:
    current_feats = selected_features + [feature]
    X_current = X[:, current_feats]
    y_pred = simple_naive_bayes(X_train, y_train, X_test)
    accuracy = np.mean(y_pred == y_test)
    if accuracy > best_acc:
        best_acc = accuracy
        best_feat = feature
```

Selected Words and Performance

Forward selection identified these 10 most relevant words:

Word	Accuracy	IDF
acting	0.500	0.693
bad	0.750	1.261
boring	0.917	1.529
action	0.917	2.485
afterward	0.917	3.401
no	1.000	2.485
all	1.000	2.996
amazing	1.000	2.708
any	1.000	2.708
audience	1.000	2.996

Key Findings

- **Intuitive Alignment**: Words match expected sentiment ("amazing"=positive, "boring"=negative)

- **IDF Pattern**: Selected words have high average IDF (2.326 vs overall 2.728), showing preference for rare but discriminative words
- **Class Differences**: Positive words tend to have higher IDF than negative words
- **Generalization**: Core sentiment words ("amazing", "bad", "boring") should work well on IMDB; domain-specific words ("afterward") may not
- **Performance**: Perfect accuracy achieved with just 6 features

The feature selection successfully identified meaningful words that achieve excellent classification performance.

AI Tools Usage Disclosure

AI Assistance

This report was prepared with the assistance of AI tools (ChatGPT) for the following purposes:

- **Code Debugging**: Assistance with identifying and fixing implementation errors in machine learning algorithms
- **LaTeX Formatting**: Help with LaTeX syntax and document structure
- **Concept Clarification**: Explanation of technical concepts including silhouette scores, HAC linkage methods, and feature normalization
- **Academic Writing**: Assistance with improving the clarity and structure of technical explanations

My Contribution

All final implementations, analysis, results, and conclusions are my own work. The AI assistance was used solely for:

- Clarifying concepts I already understood
- Debugging code I wrote independently
- Improving the presentation of my original work

Resources Used

- Course lecture notes and materials
- Textbook: Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*
- Official Python documentation for NumPy, Pandas, and Matplotlib
- UiT machine learning exercise solutions from exercise labs
- Wikipedia (many pages)